FlightFinder

Full-Stack Flight Booking Platform

Project Report

Team ID	LTVIP2026TMIDS88404
Team Size	4 Members
Team Leader	Dasari Mounika
Members	Dhanusha Avvari, G Manoj Kumar, Gumma Harika
Date	March 2026
Stack	MongoDB · Express.js · React.js · Node.js

Powered by the MERN Stack & JWT Role-Based Authentication

Confidential — March 2026

Table of Contents

1. Abstract
2. Introduction
3. Ideation Stage
4. Requirements Stage
5. Design Stage
6. Development Stage
7. Testing Stage
8. Results & Implementation
9. Future Work
10. Conclusion
11. References

1. Abstract

FlightFinder is a full-stack web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that provides a complete flight discovery and booking platform. The system implements a robust role-based access control architecture, differentiating between three distinct user personas: Traveler, Operator, and Administrator — each with a dedicated dashboard and a tailored set of features.

The platform enables travellers to search available flights, make real-time seat bookings with passenger detail capture, and manage their booking history. Flight Operators can list and manage their own routes, while Administrators maintain platform-wide oversight — managing users, flights, and bookings from a centralised control panel.

Security is enforced at every tier through JSON Web Token (JWT) authentication, bcrypt password hashing, and a middleware-based authorisation layer on all protected API endpoints. The result is a scalable, production-ready solution that demonstrates enterprise-grade full-stack engineering principles.

2. Introduction

The global air travel industry is characterised by rapidly changing schedules, dynamic seat availability, and multi-stakeholder interactions between passengers, airlines, and platform administrators. Traditional travel booking platforms are often monolithic, difficult to maintain, and lack the flexibility to cater to different user roles from a single unified codebase.

FlightFinder addresses these challenges by delivering a clean, modular MERN stack application. By leveraging React's component-driven architecture on the frontend and Express.js's lightweight routing capabilities on the backend, the system achieves a clear separation of concerns while remaining fully interconnected through a RESTful API.

A key innovation of this project is the operator approval workflow, where newly registered flight operators remain in a pending state until explicitly approved by an administrator — mirroring real-world airline onboarding pipelines and preventing unauthorised flight listings.

3. Ideation Stage

Problem Identification

Flight booking systems require careful coordination across multiple independent actors: passengers who seek convenient search and booking tools, airline operators who need efficient route and inventory management interfaces, and platform administrators who must maintain system integrity. Building a single system that satisfies all three roles without compromising usability for any of them is a significant engineering challenge.

Approach

- Architecture: Design a multi-tier MERN stack application with clearly separated client, server, and database layers.

- Role-Based Access: Implement three independent dashboards (Traveler, Operator, Admin) secured by JWT and role middleware.
- Operator Lifecycle: Build an approval workflow ensuring operators are vetted before gaining platform access.
- Booking Engine: Develop a real-time seat-selection and passenger-management booking flow with cancellation support.
- Responsive UI: Design a clean, mobile-friendly interface using a consistent design system with a blue-navy palette.

Value Proposition

FlightFinder delivers a production-quality, end-to-end booking solution that simultaneously serves the needs of individual travellers, airline operators, and platform administrators — providing a reusable, extensible architectural template for real-world travel technology products.

4. Requirements Stage

Functional Requirements

- Authentication System: Secure registration and login for all roles, with JWT issuance and bcrypt password hashing.
- Role-Gated Dashboards: Distinct, protected interfaces for Traveler (/dashboard), Operator (/operator), and Admin (/admin) users.
- Flight Management: Operators can create, view, and update their own flight listings; Admins can delete any flight.
- Booking Engine: Travellers can search flights by origin/destination, book seats with full passenger details, and cancel confirmed bookings.
- Operator Approval Workflow: Newly registered operators enter a 'pending' state; Admin must approve before operator access is granted.
- Admin Control Panel: Full visibility across all users, bookings, and flights with management capabilities.
- Protected API Layer: All non-public endpoints are secured via authentication and role-based middleware.

Tools & Technologies

Category	Technology
Frontend Framework	React.js 18 with React Router v6
HTTP Client	Axios (with JWT interceptor)
Backend Runtime	Node.js with Express.js
Database	MongoDB with Mongoose ODM
Authentication	JWT (jsonwebtoken) + bcrypt
Styling	Custom CSS — Inter font, light theme
Development Tools	Nodemon, dotenv, VS Code
Database Seeding	Custom seed.js script

5. Design Stage

The system was architected using a standard three-tier MERN structure to ensure clean separation of concerns, maintainability, and scalability. The following subsections outline the key architectural and design decisions made during this phase.

System Architecture

- Client Layer (React.js): A single-page application communicating with the backend exclusively via RESTful HTTP requests. All JWT tokens are stored in React context and injected into outgoing requests via an Axios interceptor.

- API Layer (Express.js): A stateless REST API server that handles all business logic, authentication, and database interactions. All sensitive routes are protected by authMiddleware and roleMiddleware.
- Database Layer (MongoDB): Three Mongoose collections — User, Flight, and Booking — with defined schemas, enumerations, and ObjectId references to enforce relational integrity.

Data Flow

User authenticates → JWT issued → stored in React AuthContext → Axios attaches token to all subsequent requests → Express authMiddleware validates JWT → role-specific middleware grants or denies access → Mongoose queries MongoDB → response returned to React UI.

Design System

Token	Value
Primary Colour	#2563EB (Blue)
Success	#10B981 (Green)
Danger	#EF4444 (Red)
Background	#F1F5F9 (Light Gray)
Navbar	#1E293B (Dark Navy)
Typography	Inter (Google Fonts)

6. Development Stage

Database Schema Design

Three Mongoose models form the data backbone of the application:

Collection	Key Fields	Notes
User	username, email, password, usertype, approval	Enum: traveler | operator | admin
Flight	flightName, flightId, origin, destination, departureTime, arrivalTime, basePrice, totalSeats, availableSeats, operatorId	operatorId references User
Booking	user, flight, flightName, passengers[], totalPrice, journeyDate, seatClass, bookingStatus	Enum: confirmed | cancelled

Authentication & Authorisation

- Registration: Passwords hashed with bcrypt (10 salt rounds) before storage. Operator accounts default to pending approval status.

- Login: Credentials verified against the hashed password; a signed JWT (7-day expiry) containing user ID, email, username, and usertype is returned.
- authMiddleware: Validates the Bearer token on every protected route, attaching the decoded user payload to req.user.
- adminMiddleware: An additional guard applied to Admin-only endpoints, returning HTTP 403 for unauthorised access attempts.

API Endpoints

Method + Endpoint	Auth Required	Description
POST /api/auth/register	None	Register as traveler or operator
POST /api/auth/login	None	Login and receive JWT
GET /api/flights	None	Retrieve all flights
GET /api/flights/search	None	Search flights by origin/destination
POST /api/flights	Operator/Admin	Create a new flight listing
PUT /api/flights/:id	Operator/Admin	Update an existing flight
DELETE /api/flights/:id	Admin only	Delete a flight
POST /api/bookings	Traveler	Create a new booking
GET /api/bookings/my	Traveler	Retrieve own booking history
PUT /api/bookings/:id/cancel	Traveler/Admin	Cancel a booking
GET /api/admin/stats	Admin	Retrieve platform-wide statistics
GET /api/admin/users	Admin	Retrieve all registered users
PUT /api/admin/users/:id/approve	Admin	Approve a pending operator
GET /api/admin/bookings	Admin	Retrieve all bookings platform-wide
GET /api/operator/flights	Operator	Retrieve operator's own flights
GET /api/operator/bookings	Operator	Retrieve bookings on own flights
GET /api/operator/stats	Operator	Retrieve operator dashboard stats

Frontend Architecture

- App.js: Defines all application routes using React Router v6 with ProtectedRoute wrappers enforcing authentication and role checks.
- AuthContext.js: Provides global JWT state management, exposing login(), logout(), and the decoded user object to all child components.

- api.js (Axios interceptor): Centrally attaches the Authorization: Bearer <token> header to all outgoing API requests.
- BookingModal.js: A modal component that captures seat class, passenger details, mobile number, and journey date before submitting the booking.
- ProtectedRoute.js: A higher-order component that validates both authentication status and user role before rendering the requested page.

7. Testing Stage

Unit Testing

- Verified all authentication endpoints: /api/auth/register correctly hashes passwords and enforces email uniqueness; /api/auth/login returns a valid JWT only for approved users.
- Confirmed role-enforcement middleware: requests to Admin-only endpoints with an Operator token correctly receive HTTP 403 Forbidden responses.
- Validated Mongoose schema constraints — enum violations and missing required fields return appropriate HTTP 400 validation errors.

Integration Testing

- End-to-end booking flow validated: user login → flight search → BookingModal submission → confirmation in /api/bookings/my.
- Operator approval flow tested: operator registration → pending status → admin approves → operator can subsequently log in and access protected routes.
- Cancellation flow verified: booking status transitions from confirmed to cancelled and is reflected immediately in the traveller's Bookings page.

Performance & Security Testing

- Confirmed that expired JWT tokens are rejected with HTTP 401 Unauthorized, prompting the user to log in again.
- Verified that direct URL access to /admin by a traveller role user is correctly intercepted by ProtectedRoute and redirected.
- Database seeder (seed.js) validated by confirming all pre-configured users, roles, and seeded flights are correctly persisted to MongoDB.

8. Results & Implementation

8.1 Role-Based Dashboard Summary

Role	Key Features	Route Prefix
Traveler	Search flights, book seats with passenger details, view & cancel bookings, personal stats dashboard	/dashboard, /search, /bookings
Operator	Add new flight routes, manage own flights, view bookings on own listings, operational stats overview	/operator
Admin	Platform-wide stats, approve operator accounts, manage all users, bookings, and flights	/admin

8.2 User Interface Workflow

The FlightFinder web application was designed for intuitive usability across all three user roles. The following sequence of screenshots demonstrates the complete end-to-end user workflow — from the public landing page through authentication, role-specific dashboards, and core transactional flows including flight search, booking management, and administrative operations.

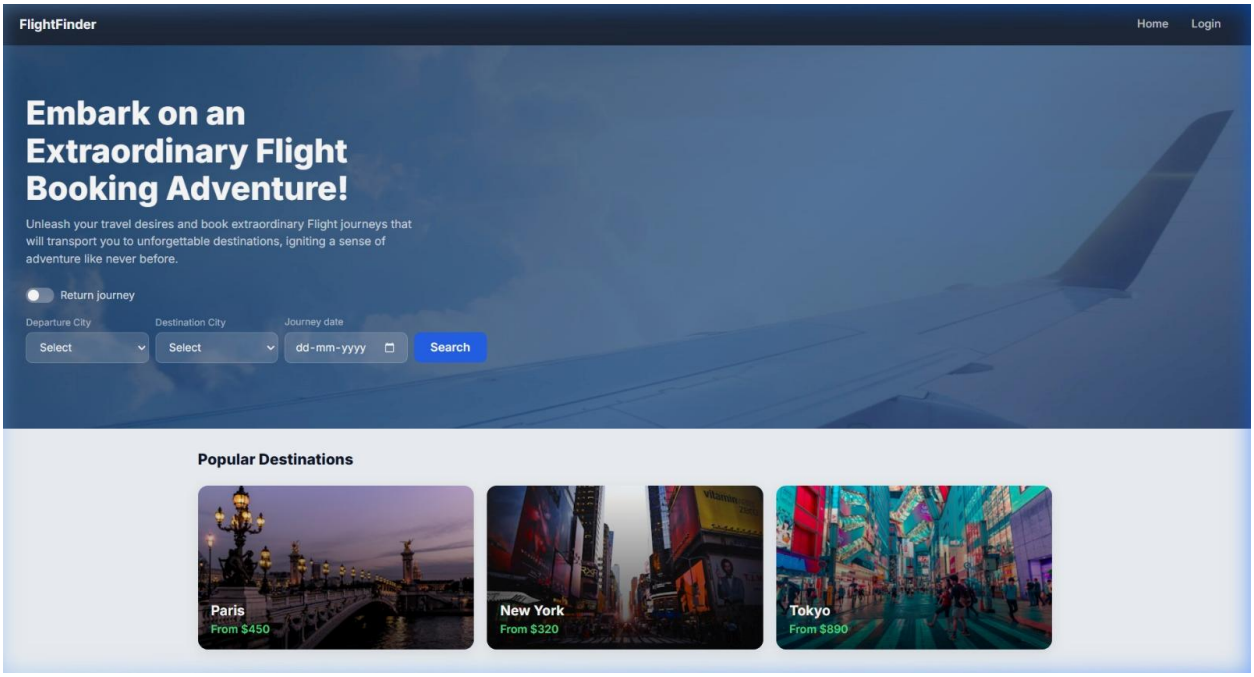

Figure 1: Landing Page — Hero section with flight search widget, departure/destination city selectors, journey date picker, and Popular Destinations showcase.

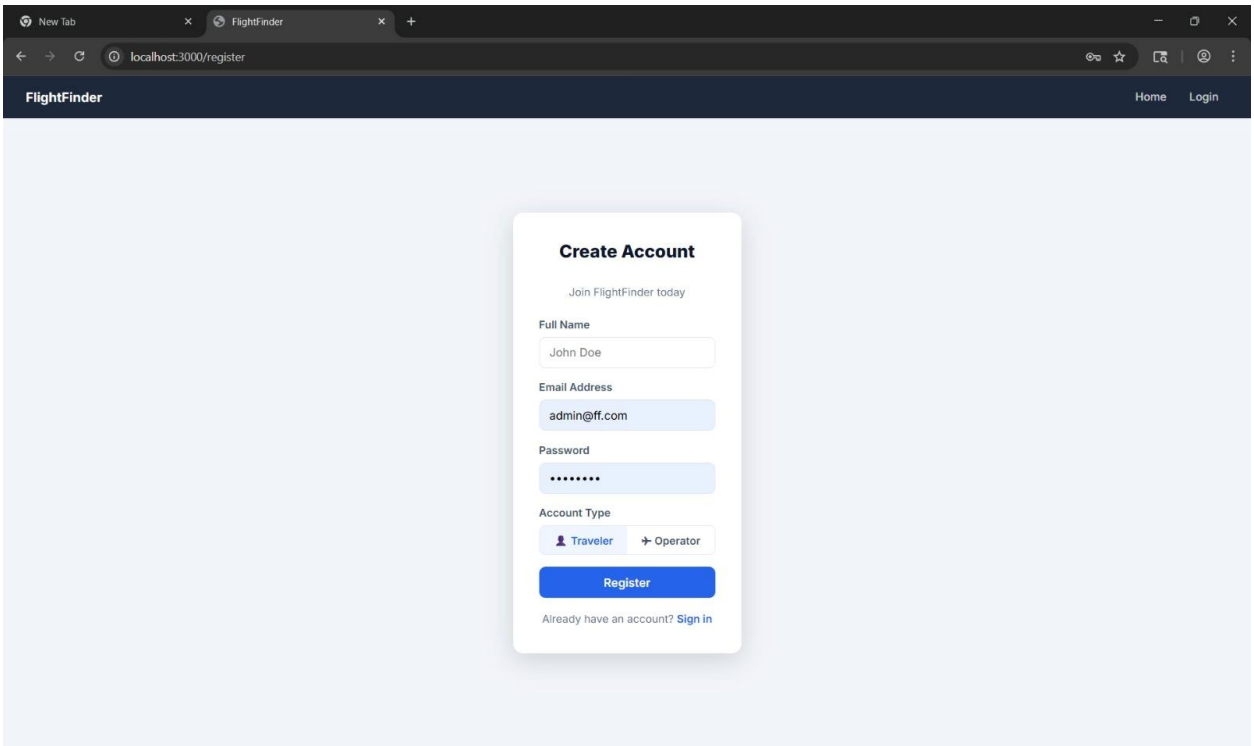

Figure 2: Registration Page (/register) — User account creation form with Full Name, Email Address, Password fields, and Account Type toggle (Traveler / Operator).

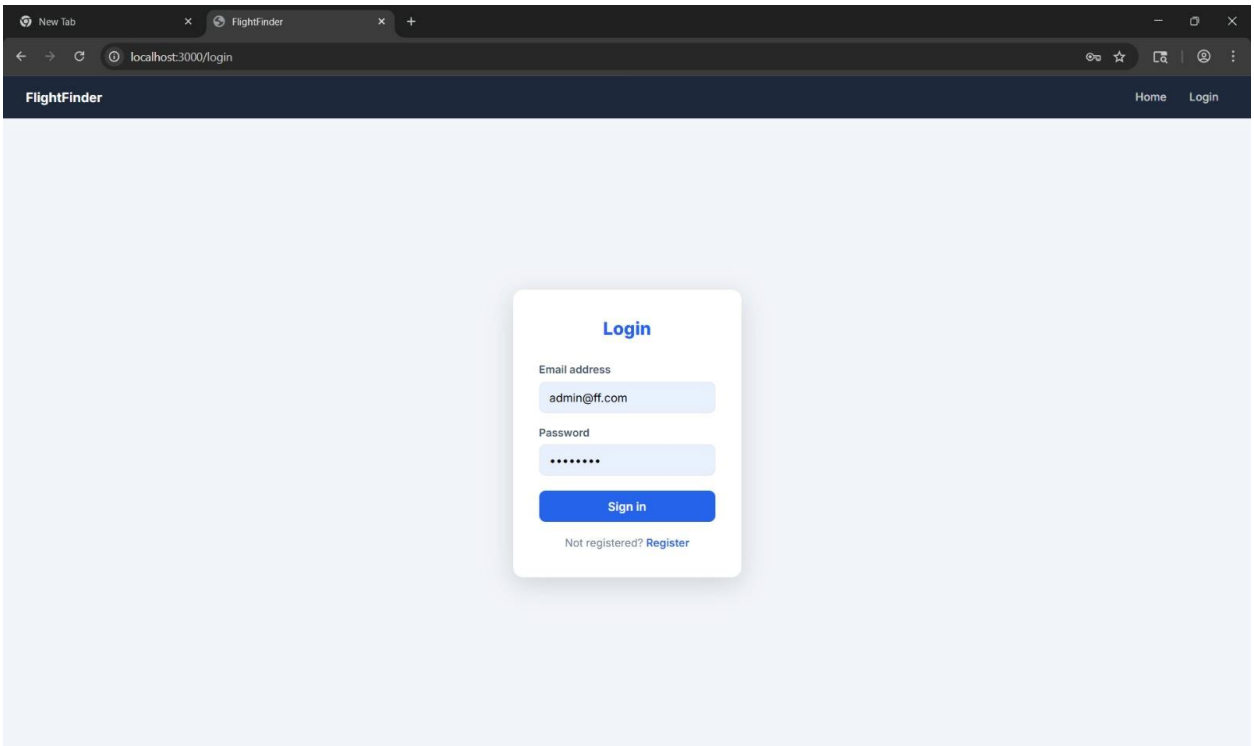

Figure 3: Login Page (/login) — Secure sign-in form accepting registered email and password credentials, with a link to Register for new users.

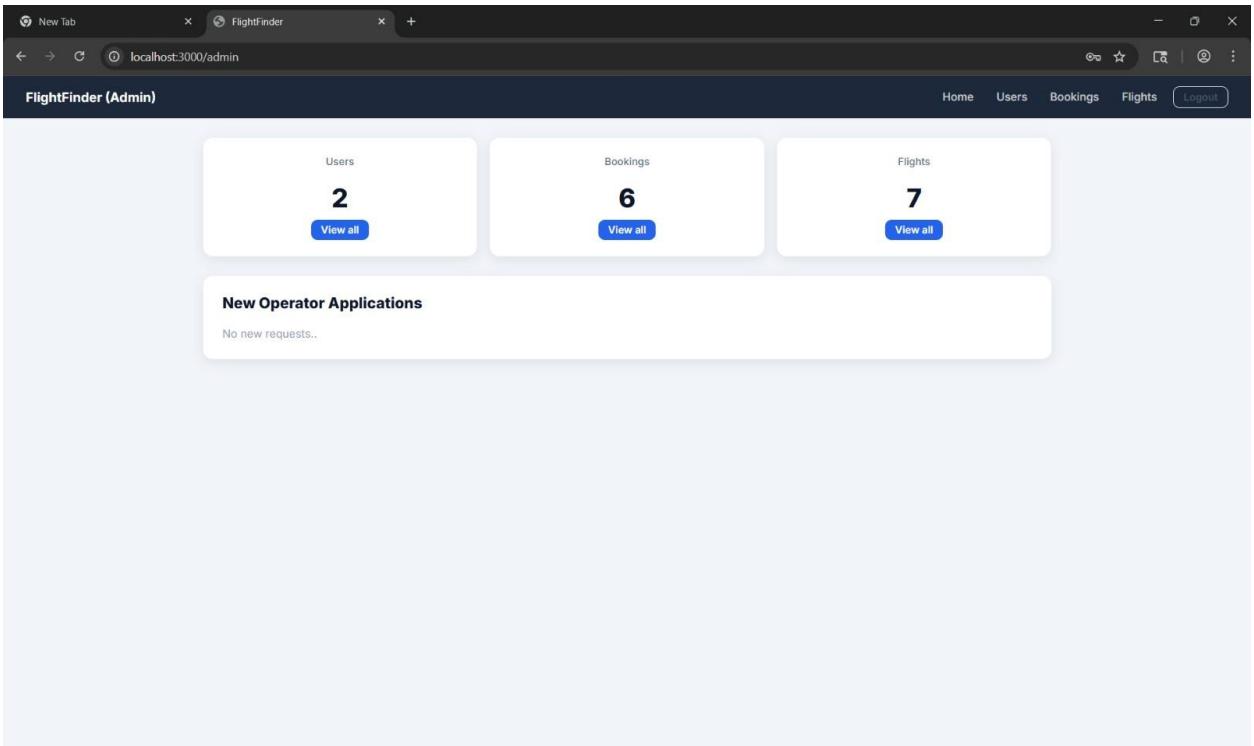

Figure 4: Admin Dashboard (/admin) — Platform overview showing live statistics: 2 Users, 6 Bookings, 7 Flights, with New Operator Applications panel for pending approvals.

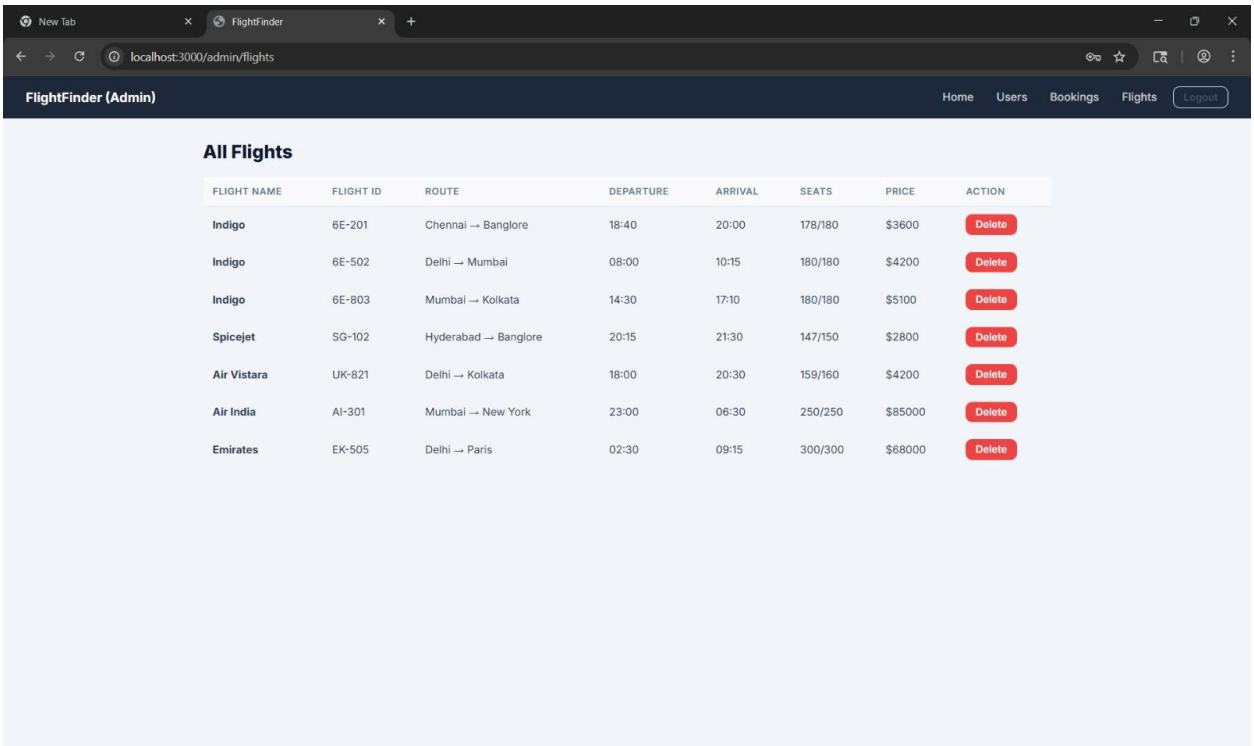

Figure 5: Admin — All Flights (/admin/flights) — Complete flight listing across all operators showing Flight Name, ID, Route, Departure, Arrival, Seat availability, Price, and Delete action.

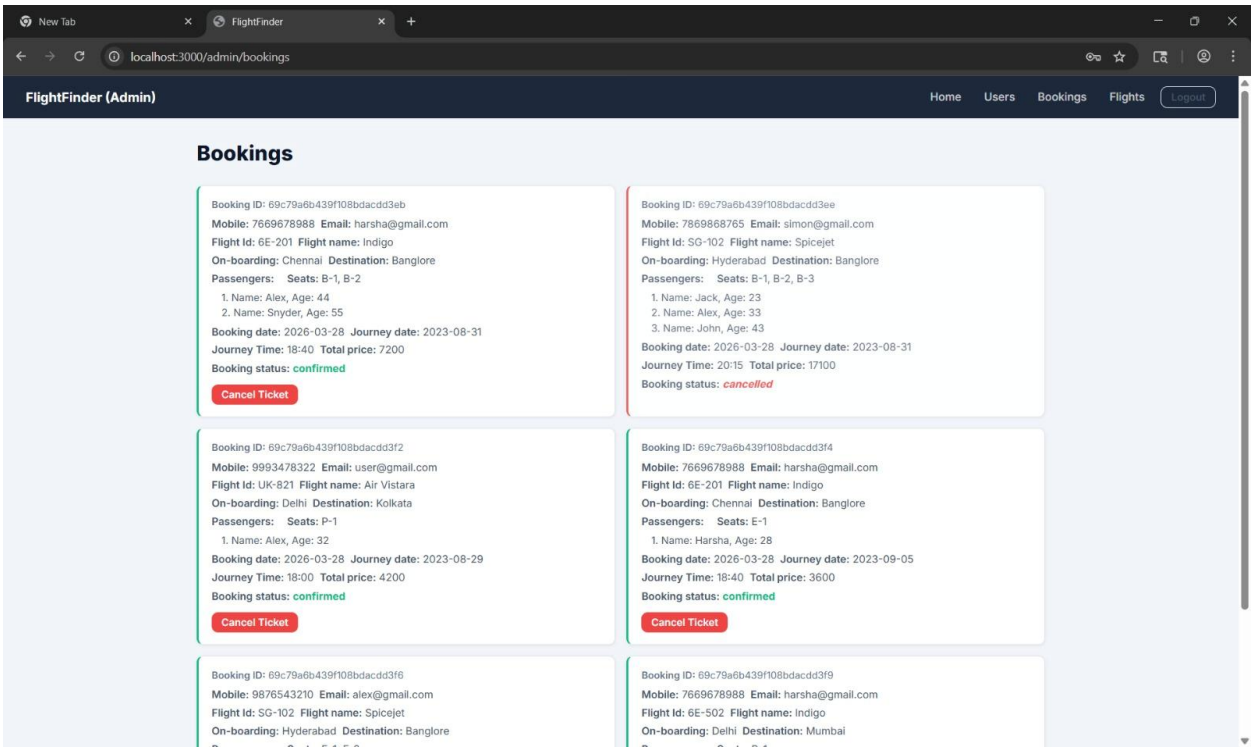

Figure 6: Admin — All Bookings (/admin/bookings) — Platform-wide booking management view displaying full booking details including Booking ID, passenger list, journey info, total price, booking status (confirmed/cancelled), and Cancel Ticket action.

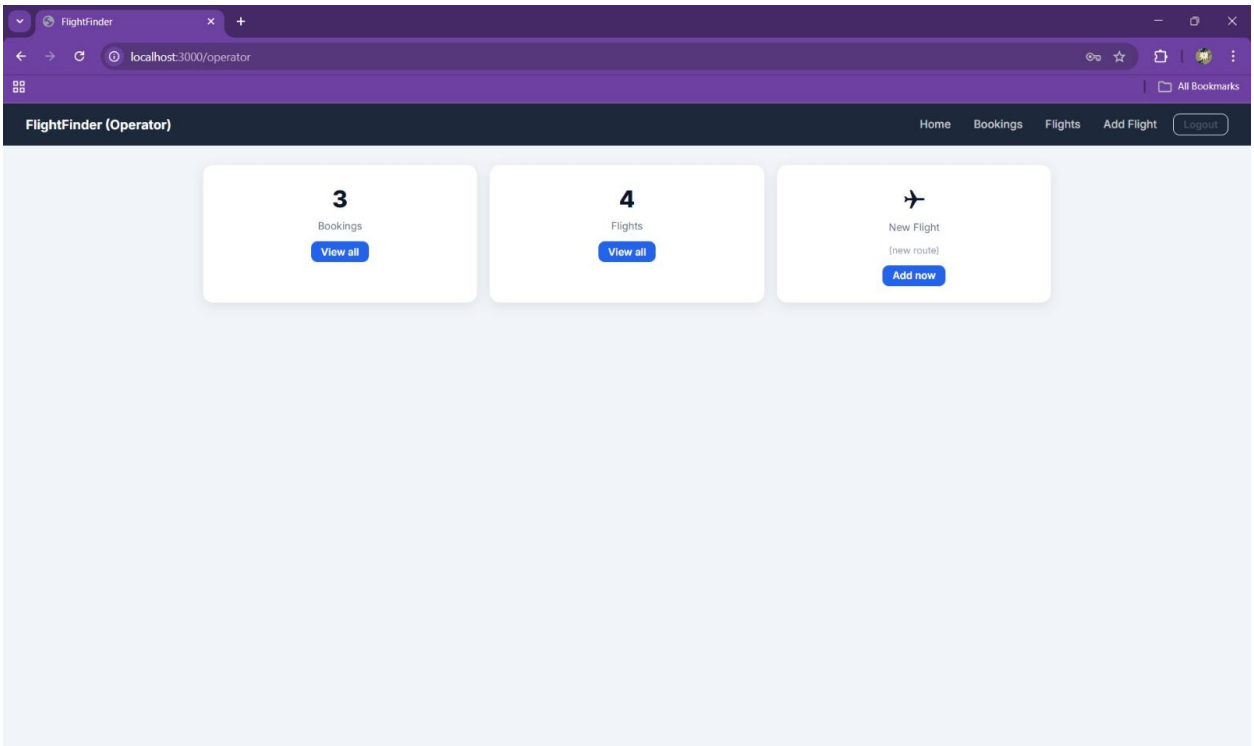

Figure 7: Operator Dashboard (/operator) — Overview panel showing 3 Bookings, 4 Flights, and a New Flight shortcut card with 'Add now' button for quick route creation.

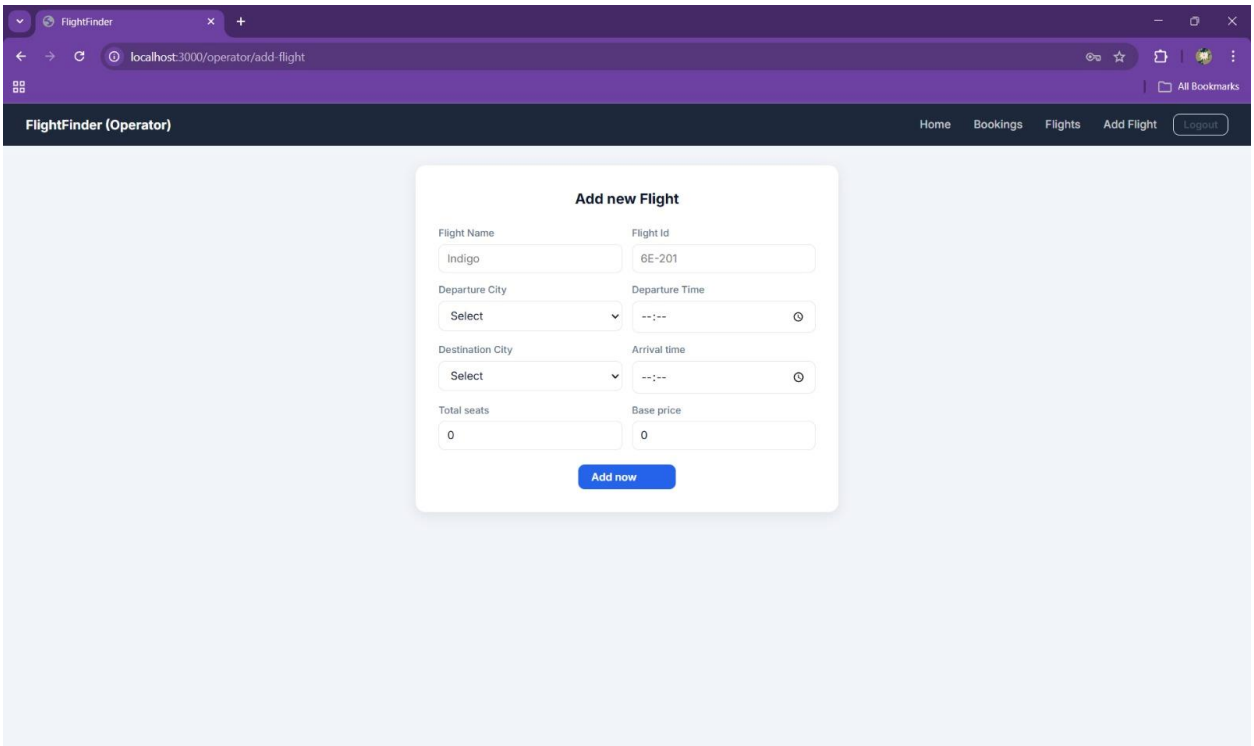

Figure 8: Add New Flight Form (/operator/add-flight) — Operator flight creation form capturing Flight Name, Flight ID, Departure City, Departure Time, Destination City, Arrival Time, Total Seats, and Base Price.

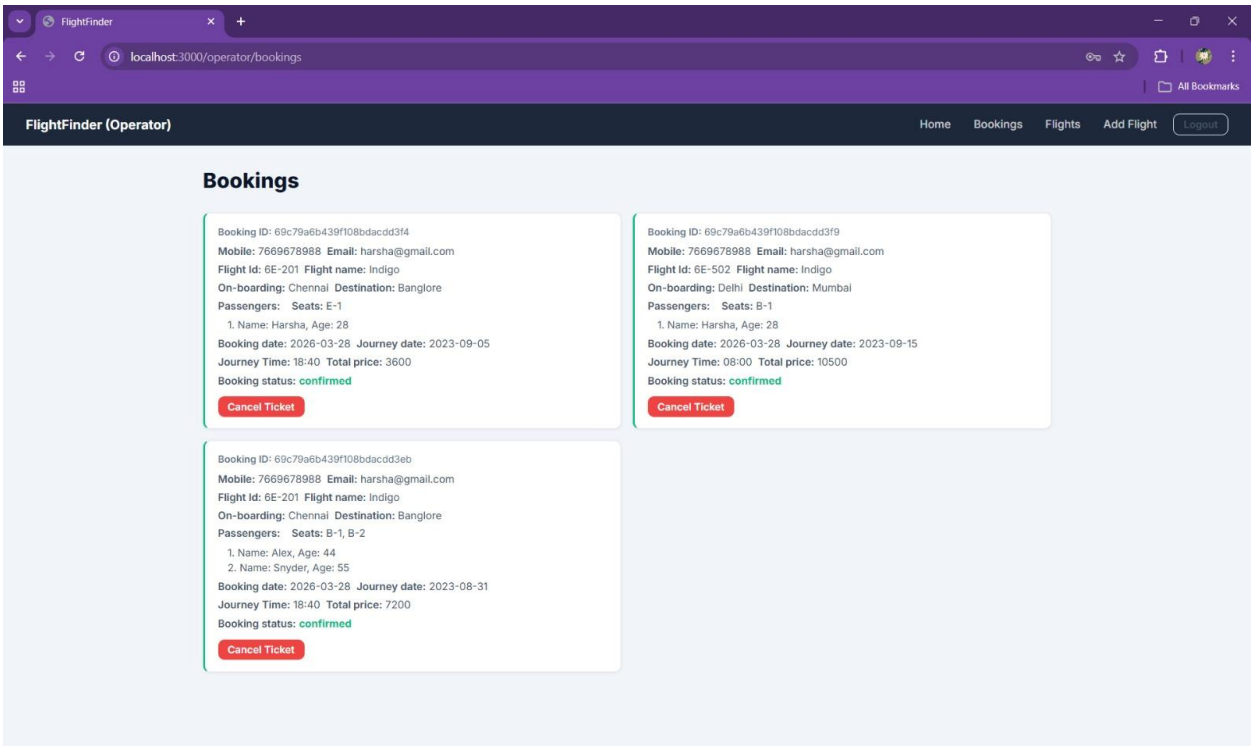

Figure 9: Operator — Bookings View (/operator/bookings) — Operator-scoped bookings showing passenger details, seat assignments, journey dates, total price, and booking status with Cancel Ticket option.

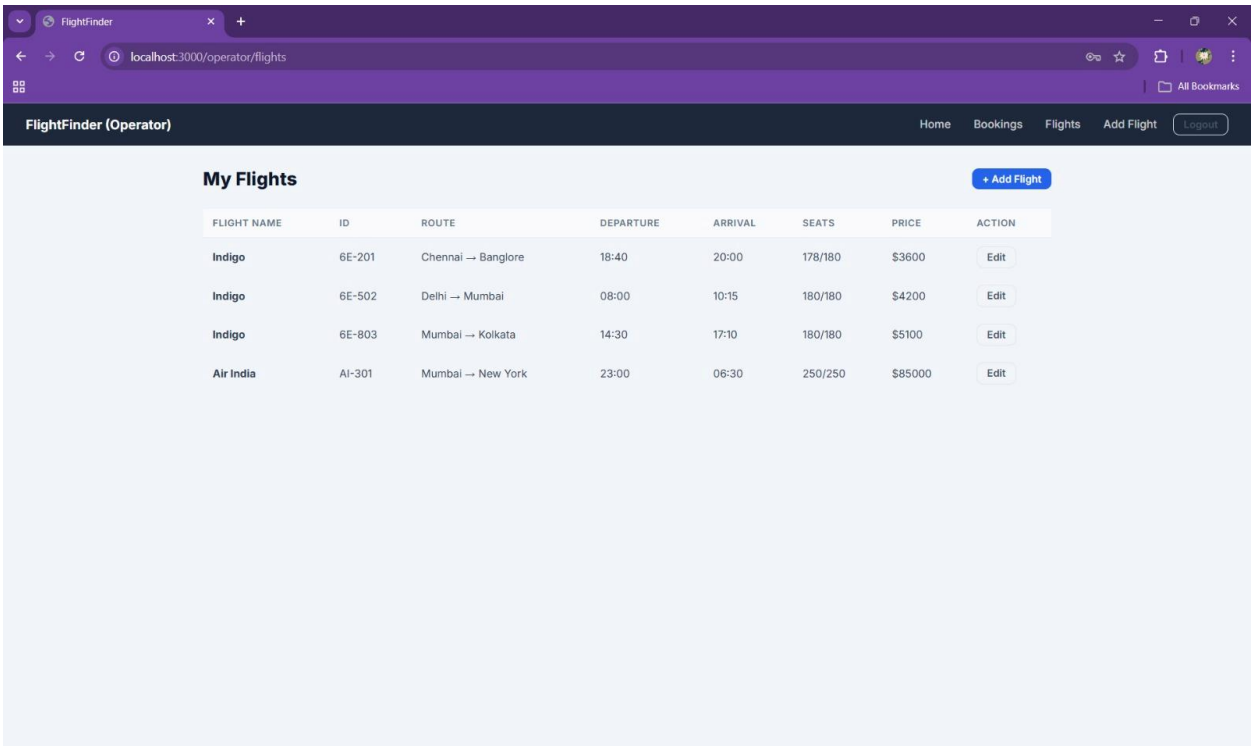

Figure 10: Operator — My Flights (/operator/flights) — Tabular view of operator's own active flights with columns for Flight Name, ID, Route, Departure, Arrival, Seats availability, Price per person, and Edit action.

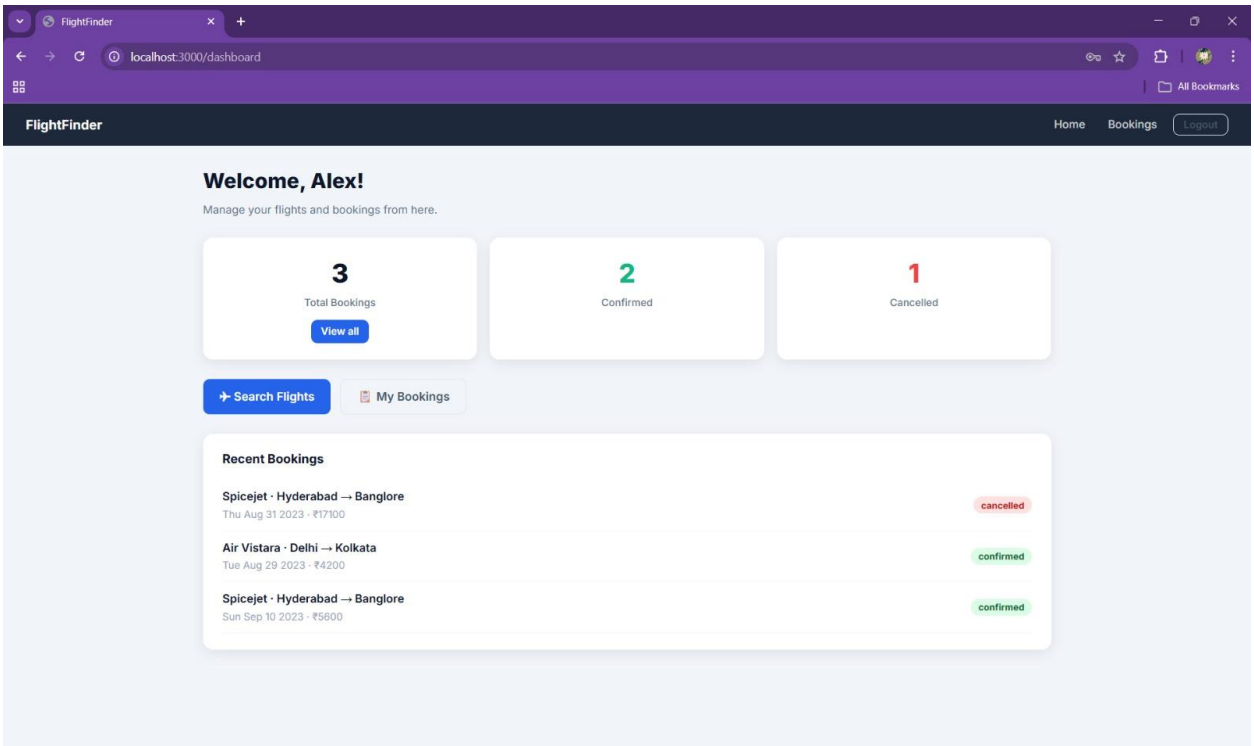

Figure 11: Traveler Dashboard (/dashboard) — Personalised welcome screen showing Total Bookings (3), Confirmed (2), and Cancelled (1) stats, Search Flights and My Bookings quick-access buttons, and a Recent Bookings summary list.

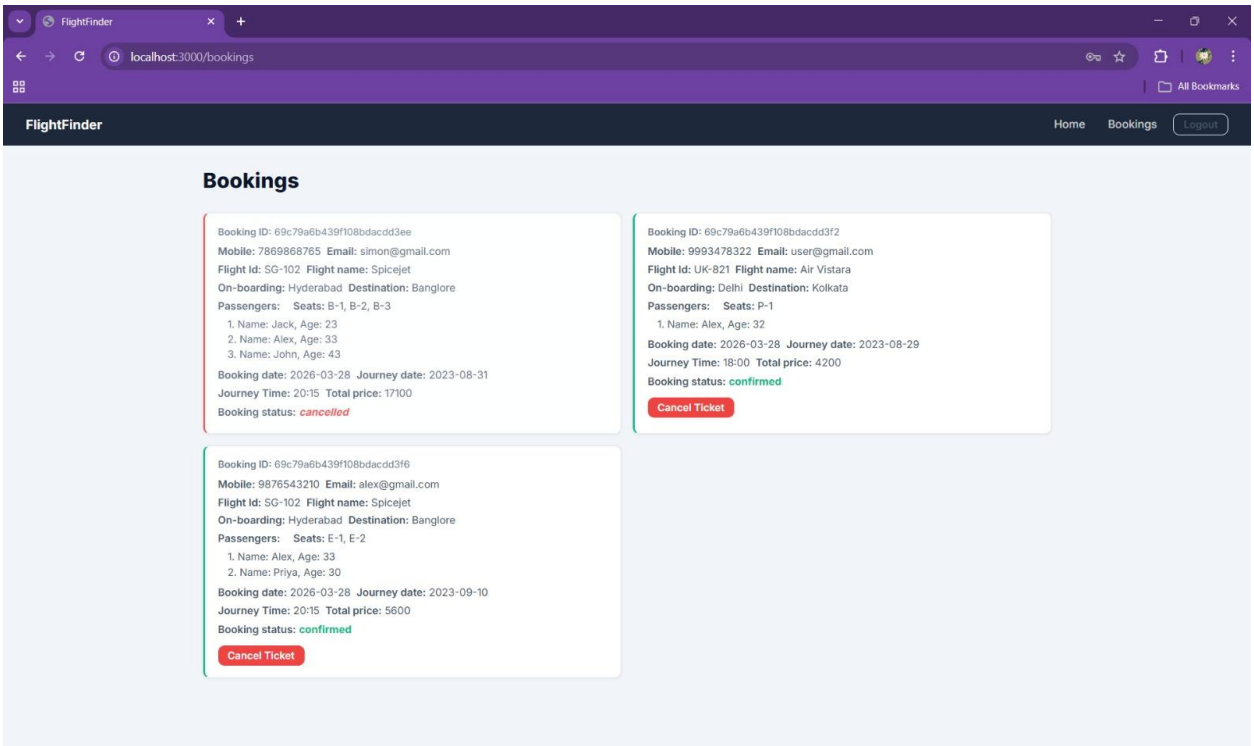

Figure 12: Traveler — My Bookings (/bookings) — Detailed booking cards for the logged-in traveller displaying Booking ID, flight details, on-boarding/destination cities, passenger names, seat assignments, journey date & time, total price, booking status, and Cancel Ticket button.

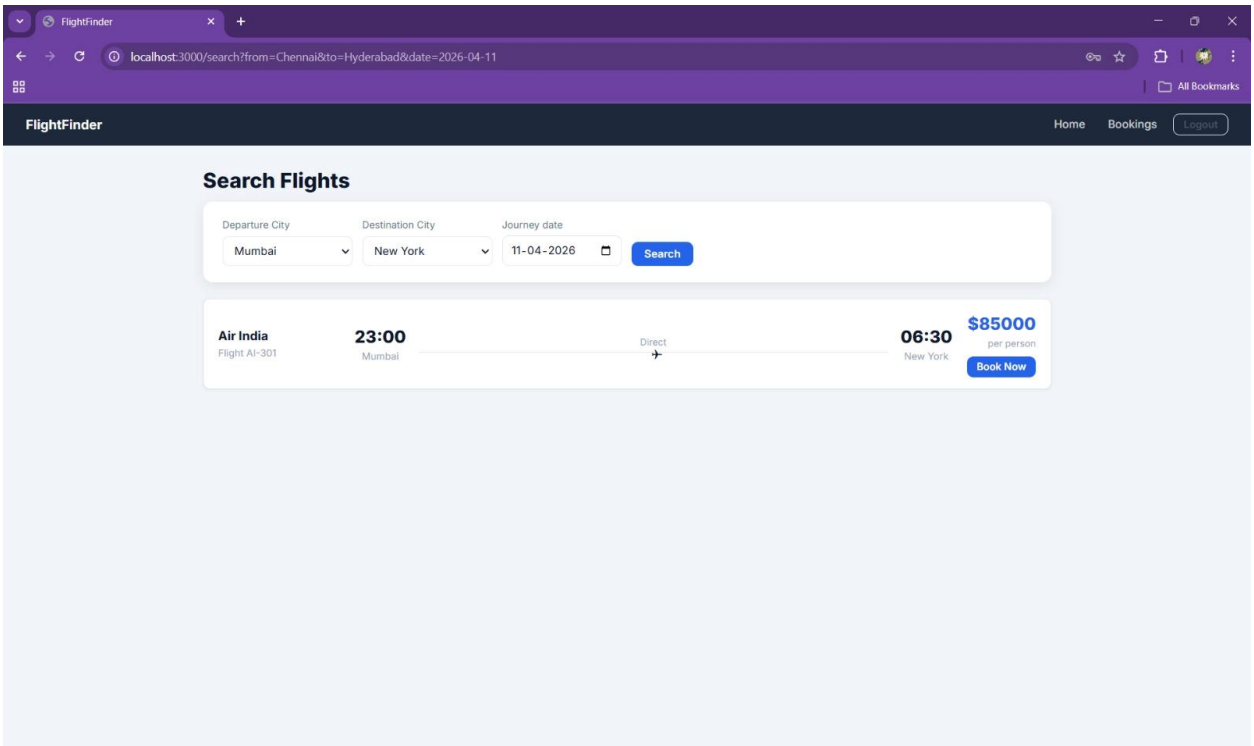

Figure 13: Search Flights (/search) — Flight search results page showing filtered flights by Departure City (Mumbai), Destination City (New York), and Journey Date, displaying Air India Flight AI-301 with departure time, arrival time, Direct indicator, price ($85,000 per person), and Book Now button.

8.3 Pages Summary

Page	Route	Visible To
Landing Page with Hero Search	/	Everyone
Login	/login	Everyone
Register	/register	Everyone
Search Flights	/search	Everyone
Traveler Dashboard	/dashboard	Traveler
My Bookings	/bookings	Traveler
Admin Overview	/admin	Admin
Admin — All Users	/admin/users	Admin
Admin — All Bookings	/admin/bookings	Admin
Admin — All Flights	/admin/flights	Admin
Operator Overview	/operator	Operator
Add New Flight	/operator/add-flight	Operator
Operator Flights	/operator/flights	Operator

Page	Route	Visible To
Operator Bookings	/operator/bookings	Operator

8.4 Seeded Test Credentials

Role	Email / Password	Redirects To
Admin	admin@ff.com / admin123	/admin
Operator	indigo@ff.com / indigo123	/operator
Operator	spicejet@ff.com / spicejet123	/operator
Traveler	alex@ff.com / alex123	/dashboard
Traveler	harsha@ff.com / harsha123	/dashboard

9. Future Work

- Payment Gateway Integration: Integrate a payment provider such as Stripe or Razorpay to enable real monetary transactions with booking confirmation receipts.
- Live Seat Map: Replace text-based seat codes (e.g., B-1, B-2) with an interactive graphical seat-selection map, dynamically reflecting real-time seat availability.
- Email Notification Service: Implement transactional email notifications for booking confirmations, cancellations, and operator approval decisions using NodeMailer or SendGrid.
- Production Deployment: Containerise the application using Docker with a docker-compose setup for frontend, backend, and MongoDB; deploy to AWS EC2 or GCP.
- Advanced Flight Search: Extend the search engine to support multi-city itineraries, flexible date search, and price-range filtering.
- Mobile Application: Develop a React Native companion app to extend the FlightFinder platform to iOS and Android, sharing the existing Express.js backend.

10. Conclusion

FlightFinder successfully demonstrates the end-to-end design, development, and deployment of a production-quality flight booking platform using the MERN stack. By establishing a clear, role-based architecture from the outset, the system cleanly serves three distinct user personas from a single, unified codebase — each receiving a tailored interface and a precisely scoped set of permissions.

The implementation of JWT authentication, bcrypt password hashing, and API-level middleware ensures that every interaction with the system is appropriately secured. The operator approval workflow adds a layer of administrative governance that mirrors real-world platform operations.

The project delivers a fully functional, extensible framework that is well-positioned for enhancement with features such as live payment processing, seat mapping, and cloud-native deployment — making it a strong foundation for a commercial travel technology product.

11. References

- GitHub Repository: Flight_Finder-main (project source code)
- MongoDB Documentation — https://www.mongodb.com/docs
- Express.js Documentation — https://expressjs.com
- React.js Documentation — https://react.dev
- Node.js Documentation — https://nodejs.org/en/docs
- jsonwebtoken (npm) — https://www.npmjs.com/package/jsonwebtoken
- bcrypt (npm) — https://www.npmjs.com/package/bcrypt
- Mongoose ODM — https://mongoosejs.com/docs
- React Router v6 — https://reactrouter.com
- Axios HTTP Client — https://axios-http.com